THE KNAPSACK EXPEDITION / C++ 学习远征

背包探险指南

从一次“拿或不拿”
走向提高组的状态设计

有限的空间里，怎样保留最好的可能?
先手推、再建模，让每一格都说得出理由。

30 个递进站点 | 6 个互动实验 | 父女共同探究

给孩子，也给陪她思考的爸爸

先学会问：这一格到底记着什么?

本讲从生活中的选择开始，慢慢推导动态规划。不预设已经学过背包、树或单调队列。基础语法是起点；提高内容按先修能力选读，一站可以拆成几次。

你会看到讲解例子与推导，也会遇到需要停下来独立完成的任务。先预测，再操作实验；先解释，再写代码。提示默认折叠，没有可直接提交的完整 C++ 题解。

第一层 · 01—10

建立模型

方案 → 状态 → 转移
旧层、新层与恰好要求

第二层 · 11—20

改变条件

次数、目标、组与维度
恢复名单与最优计数

第三层 · 21—25

提高选学

依赖与树合并
队列、位集与折半

> **第一次只走一个小闭环**
>
> 推荐先用三张卡片完成第 1—3 站中的一个完整任务。首次可用 20—35 分钟观察状态；如果能独立枚举，就继续研究状态，不反复问已经会的细节。今天不必走到一维模板，更不必读完提高部分。

爸爸先读：陪练与分层验收 · 真实题目路线 · 方法卡与空白反思

本次学习背景：2026-10-11 开始父女一起参加 ABC 479；B 题在爸爸讲解后孩子自己 AC，C 题基本没有时间看。B 的考点是最长连续段，讲义以“状态含义”连接到背包，不据此诊断具体错因。核实原题。

整条路线

01 出发：一个背包，两个不同的量

02 先试直觉：最贵、最轻、最划算可靠吗?

03 笨办法很重要：把“拿或不拿”画成树

04 状态：给一类问题贴一张准确标签

05 转移：把所有方案分成两个抽屉

06 第一次填表：每一格都说得出来源

07 从表到程序：先把二维版本写稳

08 少存几行：删掉的是存储，不是逻辑

09 循环方向：同一件物品会不会偷偷回来?

10 “至多”与“恰好”：初始化就是题意

11 只问能不能：子集和与划分

12 可以重复拿：完全背包

13 尽量少用：最少硬币与最小代价

14 数方案：顺序到底算不算不同?

15 库存有限：多重背包与二进制拆分

16 同组只能选一个：分组背包

17 有两个限制：重量与件数都要记

18 容量太大：把价值搬到横轴上

19 不仅要分数：沿着来路找回名单

20 最优方案有几种：分数与计数同时维护

21 选了孩子必须选父亲：依赖与树上背包

22 多重背包再提速：单调队列优化

23 许多真假一起算：bitset 子集和

24 件数少、数值大：折半搜索

25 提高组的第一步：看范围，选状态

26 调试：让错误在最小例子里现形

27 独立远征：三道逐层融合的任务

28 回到 AtCoder：按方法选题，不按字母猜难度

29 给爸爸：把你的理解变成她的思考空间

30 带走一张方法卡：下一题从哪里开始?

打印版包含完整正文与独立任务；互动实验请在 HTML 中操作，练习提示保持折叠。空白处由孩子自己记录。编制与题源核对：2026-10-12。

出发：一个背包，两个不同的量

这一站带走：分清资源限制与优化目标，自己比较完整方案。

先修：数组、循环、比较与加法

空间有限，想带走的东西却很多。你会怎样选?

周末去营地，每件装备只能带一件，不能切开。背包最多承受 6 单位重量。装备能带来的帮助，用“价值”表示。价值是题目给定的分数，不必与重量成正比。

装备	重量 w	价值 v	允许次数
A 观察镜	2	3	0 或 1 次
B 水壶	3	4	0 或 1 次
C 小帐篷	4	5	0 或 1 次

一个**方案**是一份完整的装备名单。总重量不超过 6，才叫**可行**；所有可行方案中总价值最大的，才叫**最优**。可以留空位，也可以一件不拿。这里不要求装满。

“拿得最多”“装得最满”“价值最高”是三件事。即使这组数据有时碰巧给出同一份名单，也不能把目标换掉。程序最后要输出最大价值；如果题目另要名单，后面还要学习怎样恢复选择。

> **统一约定**
>
> 本讲基础物品的重量是正整数，价值是非负整数；每个带编号的物品是独立的一件。变式会明确说明负收益、恰好要求、相同物品是否区分。时间、预算、体积也能充当“容量”，单位要统一。

> **停下来，轮到你想**
>
> 用上面三件装备列出你认为需要比较的全部方案。给每份写重量、价值和是否合法，再解释你选中的方案为什么没有漏掉更好的可能。

这一站的学习证据：能独立说明“重量负责判断能否放，价值负责比较好坏”，且每个方案中同一编号最多出现一次。

先试直觉：最贵、最轻、最划算可靠吗?

这一站带走：用反例识别不能贸然丢弃的选择。

先修：数组、循环、比较与加法

一个看起来很好的单件，不一定属于最好的组合。

容量 6，有 A=(4,7)、B=(3,5)、C=(3,5)，括号依次表示重量、价值。先拿价值最大的 A，剩下的空间装不下 B 或 C。单位重量价值最大也是 A，因为 7/4 大于 5/3。可是 B 与 C 能一起放进去。这说明“最贵优先”和“性价比优先”都可能丢掉更好的组合。

“最轻优先”同样不能保证价值最高：容量 4，只有 A=(1,1)、B=(4,8)。若先拿 A，B 就再也放不下。选择造成的空间碎片，会影响后续搭配。

为什么上一份贪心讲义里的分数背包又能用?

若物品可以任意切分，价值与所取比例成正比，单位价值高的部分可以逐小份交换进去。整件背包没有这个自由：不能只取半顶帐篷。一个条件变化，就可能使原来的证明失效。

> **从贪心接到动态规划**
>
> 我们还没有证明某个单独选择永远安全，就先保留不同条件下的最好结果。后面要研究的，是怎样少记很多重复历史，同时不漏掉最优解。

> **停下来，轮到你想**
>
> 给“每次先拿性价比最高的整件物品”设计一组只有三件物品的反例。画出它拿到的方案和一份更好的方案，指出空间在哪一步被占住了。

这一站的学习证据：能用合法数据反驳一条规则，同时知道一个失败例子不能推出“所有选择题都不能贪心”。

笨办法很重要：把“拿或不拿”画成树

这一站带走：看懂穷举为什么可靠，以及为什么会重复工作。

先修：数组、循环、比较与加法

先有一个一定不漏的办法，再想怎么让它省事。

回到三件装备。对 A 作两种决定；每种决定下再对 B 作两种决定；最后决定 C。三层以后有 $2^3=8$ 个叶子，每个叶子就是一份名单。无论以后用多聪明的算法，它都应该与这些合法叶子中的最佳值相同。

决定深度	已做的事	还没做的事
0	还没有决定	A、B、C
1	A 拿 / 不拿	B、C
2	A、B 的决定已定	C
3	名单已完整	核对重量与价值

递归只是画这棵树的一种写法。若还不会递归，先画纸面分叉，或把 000 到 111 当作三盏开关。不要让递归语法挡住“每件只有两个选择”这条主线。

n 件有 2^n 份名单。20 件约一百万份，40 件超过一万亿份。若每份再扫描 n 件算总和，需 $O(n\cdot 2^n)$；沿树边维护重量和价值，可将常规枚举做到 $O(2^n)$ 量级。即便如此，指数增长仍很快。

我们究竟重复了什么?

走到同一件待处理物品时，不同的过去可能留下相同的剩余容量。后面的物品与可用空间完全一样，后续问题也就一样。动态规划要合并的是这样的重复问题，不是随便把两个数相同的方案混在一起。

> **停下来，轮到你想**
>
> 画三件装备的选择树。再假想有两条不同历史，都走到“只剩 C、还有 4 单位空间”，解释接下来能做的选择是否相同。

这一站的学习证据：能不漏不重地枚举三个开关，能指出两份后续问题相同需要哪些条件。

状态：给一类问题贴一张准确标签

这一站带走：理解为什么要记“处理到哪里”和“容量是多少”。

先修：数组、循环、比较与加法

状态不是一个神秘公式，而是一句说得完整的问题。

用 `F[i][c]` 表示：**只允许从前 i 件物品中选择，总重量不超过 c，能得到的最大总价值。**i 与 c 是问题的标签；格子里存的价值，是这个问题的答案。c 是允许使用的上限，不是某份方案已经使用的重量。

为什么不只存“目前最高价值”？假设两份已选名单分别用了 2、4 单位重量，价值分别是 3、5，总容量 6。现在又来一件 (4,8)。原先分数低的那份留出了足够空间，能接上新物品；原先分数高的那份却不能。不同空间条件下的结果都可能有用。

为什么 i 也要记？同样容量 4，只能用 A，与允许用 A、B、C，答案可能不同。若没有物品范围，就无法判断某件是否已经用过。

把重复子问题归成状态，把每个答案算好保存，再按依赖组合出大问题，叫**动态规划**（Dynamic Programming，简称 DP）。我们不是保存所有名单，而是保存对后续有用的信息。

什么时候可以把历史忘掉？

在同样的已处理范围内，两份历史如果实际耗重相同，那么剩余物品、剩余容量都相同；价值低的历史不会更有利。给它们接上同一份合法后续，价值差仍不变。只保留较好的一份，不会失去最优答案。容量上限表则汇总该上限内所有合法历史中的最佳值。

> **这就是状态充分性**
>
> 忘记的细节不能改变未来的合法选择或收益规则。如果“选 B 必须先选 A”、颜色不能重复、下一件价格由上一件决定，只记重量可能就不够。状态设计必须重新检查。

> **停下来，轮到你想**
>
> 用完整中文解释 F[2][4]，然后设计两份“目前价值更高，但留下空间更少”的历史，说明为什么不能只保留一个全局最大值。

这一站的学习证据：能区分状态标签与格子数值；能解释合并历史要依赖哪些题目条件。

转移：把所有方案分成两个抽屉

这一站带走：从最后一件是否被选，推导 0/1 背包并证明不漏。

先修：数组、循环、比较与加法

求一个大问题时，先把它拆成两个更小、已经知道答案的问题。

现在求 F[i][c]。任何合法方案，要么不含第 i 件，要么含第 i 件。这两类互不重叠，也没有第三类。先看不拿：只从前 i−1 件里选，容量仍是 c，最佳值是 F[i−1][c]。

再看拿：第 i 件占 w_i 空间，带来 v_i 价值。把它从名单里暂时取出来，余下名单必须来自前 i−1 件，重量至多 $c-w_i$。余下部分最好能达到 F[i−1][$c-w_i$]。因此这类的最佳值是它再加 v_i。只有 $c \geq w_i$ 时，这个抽屉才存在。

> 不拿：F[i−1][c]
>
> 拿：F[i−1][$c-w_i$] + v_i（仅当 $c \geq w_i$）
>
> F[i][c] = 两个合法候选的较大值

为什么余下部分也可以取最优?

如果某份最优名单包含第 i 件，但余下部分比 F[i−1][$c-w_i$] 差，就把余下部分换成那个更好的名单。物品范围没变、重量仍合法，加回第 i 件后总价值更高，与“原名单最优”矛盾。

反过来，从任一合法的小问题方案出发，不拿或加上第 i 件，都能组成大问题里的合法方案。所以这里既没漏掉合法方案，也没凭空造出非法方案。最后用 i 从小到大计算，依赖的上一层总是先有答案。

> **0/1 的名字**
>
> 对每一件物品，选择次数只有 0 或 1。“选”的来源必须是前 i−1 件；如果来源已允许第 i 件，就有再次使用它的风险。

> **停下来，轮到你想**
>
> 不看公式，用“从名单里取出第 i 件”的方式解释：为什么拿它以后查的是前 i−1 件、容量 $c-w_i$？再说明 $w_i > c$ 时应当保留哪类方案。

这一站的学习证据：能解释两个分支为什么覆盖全部可能，以及为什么重复用第 i 件不合法。

第一次填表：每一格都说得出来源

这一站带走：把状态、转移、初值和答案位置连成完整过程。

先修：第 4—5 站

表格里的每一个数，都要能回答“你从哪里来”。

仍用 A=(2,3)、B=(3,4)、C=(4,5)，容量 6。先看 i=0：没有物品，不管容量上限是几，都可以选择空名单，最大价值为 0。随后一行一行加入物品。

可用物品 / c	0	1	2	3	4	5	6
无物品	0	0	0	0	0	0	0
只用 A	0	0	3	3	3	3	3
可用 A、B	0	0	3	4	4	7	7
可用 A、B、C	0	0	3	4	5	7	8

例如 F[2][5]：不拿 B 时沿用上一行容量 5 的结果 3；拿 B 时，上一行容量 2 的结果是 3，再加 B 的价值 4，成为 7。两类比较，保留 7。注意并没有把上一行容量 5 的 3 再直接加上 4，那会忘记先给 B 腾出空间。

最终读 F[3][6]，因为这格的问题正好是“全部三件、容量不超过 6”。F[3][5] 回答的是更小容量的另一个问题。对这种“至多”状态，容量上限增大时，可行名单只会增加，因此同一行不应下降；这可以当作检查线索。

> **表格是讲解例子，独立任务要换数据**
>
> 不要用“能抄出这一张表”检验掌握。自己换重量和价值后，仍能解释每一格的前驱，才说明懂了转移。

> **停下来，轮到你想**
>
> 另用 A=(1,2)、B=(3,5)，容量 4，自己画出全部三行。选一格，分别给“不拿”和“拿”的来源画箭头，再指出最后应读哪一格。

这一站的学习证据：能从状态句推导初值、算出整张小表，而不只会算孤立的一格。

从表到程序：先把二维版本写稳

这一站带走：建立变量职责，按依赖顺序实现并自测。

先修：第 4—6 站；基础 C++ 数组与循环

代码只是让电脑按你已经讲清的规则填表。

变量	现实含义	变化规则
n、W	物品总数、背包总容量	读入后固定
w[i]、v[i]	第 i 件物品的重量、价值	读入后固定；本讲从 1 编号
i	当前允许到第几件物品	从 1 到 n，逐层增加
c	这次计算的容量上限	枚举 0 到 W
F[i][c]	这类问题的最佳价值	由上一行的合法候选求出

```
// 这是任务骨架，不是可提交的完整程序。
准备大小为 (n+1) × (W+1) 的表
根据“无物品”解释并填写第 0 行
对每一件物品 i：
    对每一个容量上限 c：
        先保留“不拿”的结果
        若这件物品放得下：
            求“拿”的候选，并比较
输出与原题问法完全一致的那一格
```

二维版本中，计算当前行只读上一行，所以当前行容量正着或倒着计算都可以。以后压缩数组时才需要重新考虑方向；不要过早背口诀。

如果 n=100、W=100000，状态约一千万个。每格 8 字节，整张表约 80 MB，还要算容器开销；只存两行则小得多。价值和可能超过 `int`，例如若干件价值 10^9 的物品相加。先用数据范围估计总和，再决定是否使用 `long long`。

写完后的三类测试

① 关键区分：只有一件 (2,3)、容量 4，检查是否偷偷重复使用。② 边界：物品恰好等于容量，或所有物品都超重。③ 最小：一件物品、容量很小。教学允许容量 0；提交时以题面范围为准。

停下来，轮到你想

遮住前面的转移式，先在纸上写状态句和计算顺序，再独立完成二维程序。用三类测试核对，每次发现偏差先找第一格出错的位置。

这一站的学习证据：能独立写出、解释、测试二维版本；爸爸只听思路、记录提示，不接管键盘。

少存几行：删掉的是存储，不是逻辑

这一站带走：从两行滚动到一维，认识数组中的时间版本。

先修：第 7 站

上一行之后的更早历史，用完后可以回收。

算第 i 行时，只需要第 i−1 行。因此可以用 `old` 和 `next` 两行：读 old，写 next；整行完成后交换。下一轮 old 的含义变成“前 i 件”的答案。处理每格时都先从 old 的“不拿”结果开始，避免 next 留下旧垃圾。

两行滚动把 O(nW) 的空间降为 O(W)，时间仍是 O(nW)。物品编号 i 虽然不再体现在数组下标中，却仍在外层循环里控制“当前是哪一层”。

如果两行挤进同一个数组，会发生什么？

写完一个格子后，它已是新一行；没写的格子仍是旧一行。于是同一数组里暂时混着两个时间版本。读 `dp[c-w]` 时，不仅要问位置是否越界，还要问：这个位置现在属于上一轮，还是已经属于本轮？

这一问决定了物品能使用几次。一维优化必须保持原递推的依赖关系；不能只把 `F[i][c]` 的 i 删掉，认为程序自然仍对。

> **先做对，再压缩**
>
> 第一次学习推荐先写二维或两行版本。等能解释每个来源，再独立推导一维方向。空间压缩是一项需要证明的修改。

> **停下来，轮到你想**
>
> 画两排 0 到 6 的格子。计算某一件物品时，给读的位置写“旧”，写的位置写“新”；再想只留一排时，怎样避免提前覆盖还要读取的旧值。

这一站的学习证据：能说清旧行、新行各允许哪些物品，理解时间复杂度并未因压缩减少。

循环方向：同一件物品会不会偷偷回来?

这一站带走：通过不变量推导 0/1 背包倒序，而非背口诀。

先修：第 8 站

一条循环的方向，可能把“一件”变成“无限件”。

只有一件物品，重量 2、价值 3，容量 4。初始 dp=[0,0,0,0,0]。若容量从小到大，先把 dp[2] 改为 3；到 c=4 时，又读取已经更新的 dp[2]，候选变成 6。这个 6 代表把同一件物品拿了两遍。

改为从 W 倒着走到 w。计算 c=4 时，位置 2 还没处理，读到的是上一轮的 0；到 c=2 时，位置 0 也仍符合上一轮语义。这一轮每个“拿”的候选都来自不含本件物品的旧行。

> 处理容量 c 之前：大于 c 的格子已更新；不大于 c 的格子仍属上一轮。
>
> 因为 w>0，所以 c−w<c；来源格仍是旧值。

循环的下界是 w，而不是 0；否则 c−w 会成为负数。反过来，上界从 W 开始，才包含正好用到全部容量的情况。记住的是“读哪个版本”，倒序只是这个依赖关系的一种实现。

> **不能把倒序推广到所有 DP**
>
> 两行写法不怕当前行被改写；完全背包需要读取本轮；分组背包还要防止同组串用。每次都回到状态和依赖图判断。

> **停下来，轮到你想**
>
> 先预测正序与倒序分别会得到什么，再单步验证。接着关闭实验，用“处理 c 前哪些格子还是旧值”完整解释倒序为何不会重复拿。

这一站的学习证据：能独立给出单件反例与倒序不变量；修改重量后仍能解释。

“至多”与“恰好”：初始化就是题意

这一站带走：区分合法空方案与根本不存在的状态。

先修：第 5—9 站

0 分可能是一个真实结果，也可能是你误造出来的结果。

“重量不超过 c”允许留空，所以无物品时每个容量上限的最佳值都为 0。“重量恰好等于 c”则不同：没有物品只能凑出重量 0，任何正重量都凑不出来。

重新命名 `E[i][c]`：从前 i 件中选，**总重量恰好 c** 的最大价值。E[0][0]=0；E[0][c>0] 是不可达。转移仍分类为拿与不拿，但只有来源可达，才允许加上物品价值。

状态	无物品时 c=0	无物品时 c>0	题目答案
至多 c 的最大价值	0	0	F[n][W]
恰好 c 的最大价值	0	不可达	E[n][W]；若不可达则无解
用恰好表解至多问题	0	不可达	所有 0≤c≤W 的可达 E[n][c] 的最大值

只有 (2,3) 一件物品，目标重量 3。恰好 3 应是无解；若把每格初始化为 0，程序会从“重量恰好 1、价值为 0”的假状态转移，谎称能得到价值 3。

最大值问题可以用足够小的 `NEG` 表示不可达，例如在已核实数值范围后用 `-(1LL<<60)`。不要拿最小整数直接相加；先判可达再计算。允许负价值、又必须恰好选够时，最优值也可能是负数，不能用“<0 就无解”判断。

> **0 容量的意义**
>
> 在本讲正重量前提下，恰好 0 只有空名单，价值 0。若真实题目允许零重量物品，需要重新检查转移与计数；不能直接把正重量下的论证照搬。

> **停下来，轮到你想**
>
> 为只有一件 (2,3)、目标 3 的问题，分别写出“至多”和“恰好”的初始行。指出全零初始化在恰好模型中虚构了哪一个来源状态。

这一站的学习证据：能从状态句推出初值、不可达标记和答案位置，而非根据题型记一串初值。

只问能不能：子集和与划分

这一站带走：将最优值换成可达性，识别背包的资源维度。

先修：第 10 站

有时不需要比较好坏，只需要知道一个目标能不能凑出来。

给出正整数卡片 $a_1 \ldots a_n$，每张最多选一次，问能否凑出和 S。定义 `R[i][s]`：前 i 张中能否选出总和恰好 s。R[0][0] 为真，其余为假。不拿沿用上一行；拿需要上一行的 $s-a_i$ 可达。两种来源用“或”合并。

这仍是背包：数字相当于重量，S 相当于容量，只是格子从最大价值换成了真假。时间 O(nS)，空间可压到 O(S)，一维仍须保持 0/1 的旧层来源。

换一道没有“包”的题

把全部正整数分到两组，让两组总和之差尽量小。设总和 T，一组的和为 s，另一组就是 T−s。交换两组名字不影响差，所以只需考虑 $s \le T/2$；此时差为 T−2s。只要找到这个范围内最大的可达 s。

这里不能只做“平均分”：接近一半的数值不一定能由实际卡片凑出。也不能先算好目标就宣布可行；背包表正是用来确认哪些总和真实存在。

> **前提变化**
>
> 如果卡片有负数，取一张后总和未必增加；0…S 的截断可能丢解，索引也需要偏移或其他存储。先确认输入非负与正数条件。

> **停下来，轮到你想**
>
> 用卡片 [2,3,7] 列出可达总和，再把它们分成两组。请解释为什么只搜索总和的一半就够，而不是直接假定一半可达。

这一站的学习证据：能把实际题目翻译成“前几张、恰好多少、能不能”，并写出正确空状态。

可以重复拿：完全背包

这一站带走：从枚举数量到当前层依赖，推导正序。

先修：第 9—10 站

同样的装备现在有无限库存，改变的是哪条规则?

每种物品可以拿 0、1、2...件，只要放得下。直接做法枚举数量 k：从前 i−1 种在容量 $c-k\cdot w_i$ 的最佳值，加 $k\cdot v_i$。这样正确，但每格可能试很多 k。

把“至少一件”拿走一件

最优方案仍分为“不拿第 i 种”和“至少拿一件第 i 种”。第二类拿走一件后，剩下的名单里**还可以有第 i 种**，所以来源变为 F[i][c−w_i]，而不是 F[i−1][c−w_i]。

> $F[i][c] = \max(F[i-1][c], F[i][c-w_i]+v_i)$
> 第二项仅在 $c \geq w_i$ 时存在。

因为 $w_i>0$，当前层较小容量的问题更早完成，依赖没有绕成圈。压到一维时容量从小到大，让来源已经是本轮的值，于是能够继续拿同一种。

再看一件 (2,3)、容量 4。第 9 站中正序得到的 6 对 0/1 是错误的，对无限库存却是合理的。不能脱离题意，只凭某个输出更大就判定程序更好。

> **库存与前提**
> “不限次数”不等于每种必须拿。若零重量且价值为正又可无限拿，最优值将无上界；本章正重量假设排除了这种情况。混合背包也要逐种确认次数限制，再决定它的更新方式。

> **停下来，轮到你想**
> 从“至少拿一件”的方案中移走一件，解释剩下为何仍可使用当前种类。然后在同一组小数据上分别实现 0/1 和完全版本，预测哪个结果可能不同。

这一站的学习证据：能从物品次数推导来源层，再从来源层推导方向。

尽量少用：最少硬币与最小代价

这一站带走：同时改变目标、初值和合并方式。

先修：第 10—12 站

一格里不一定放“最多价值”，也可以放“最少花费”。

有若干正面值硬币，每种无限，必须恰好凑出 S，求最少枚数。定义 `best[s]` 为恰好凑出 s 的最少硬币数。best[0]=0，因为不拿硬币就凑出了 0；正金额初始化为无穷大，因为尚未找到凑法。

若最后放入面值 a 的硬币，之前必须恰好凑出 s−a，总枚数增加 1。枚举所有可能的最后一枚，取最小候选。对这个一维最少值模型，金额递增能让较小金额先算好；按种类做完全背包也可以，关键是你为每种顺序给出的状态定义。

> 候选 = best[s−a] + 1（要求 s≥a，且来源可达）
>
> best[s] 取所有合法候选中的最小值。

与最大价值相比，改变的不只是 `max` 换成 `min`。无解状态要从负无穷换成正无穷，新增收益变成新增代价，输出时也要处理目标不可达。

面值 1、3、4，金额 6，是检验“每次用最大面值”是否可靠的好例子。面值 4、6，金额 5，则能检验无解。空金额 0 能检查初值。统一用一个很大的有限数模拟无穷，并在加法前检查可达与范围。

> **停下来，轮到你想**
>
> 为“面值 4、6，恰好凑 5”写出 0 到 5 的状态含义和初值。解释为什么未访问的正金额不能填 0，再用另一组可达金额手推一次。

这一站的学习证据：能随目标一起调整状态值的含义、转移、初值和无解输出。

数方案：顺序到底算不算不同?

这一站带走：理解计数对象、唯一来源与循环顺序。

先修：第 11—13 站

“一枚 1 再一枚 2”和“一枚 2 再一枚 1”，是不是两种方案?

先把方案等价规则写清。若只看各种硬币用了几枚，顺序不重要，1+2 与 2+1 是同一组合。若记录依次放入的序列，它们就是两个不同序列。无限供应面值 1、2，目标 3：组合是 {1,1,1}、{1,2}；序列还要把 {2,1} 单独计入。

组合：按种类分层

令 C[i][s] 为只用前 i 种凑出 s 的组合数。不用第 i 种，来自 C[i−1][s]；至少用一枚，拿走一枚后来自 C[i][s−a_i]。这两类互斥，所以相加。空金额有一种空组合，因此 C[0][0]=1。压缩后“种类在外、金额递增在内”保持了这个分类。

序列：按最后一个元素分类

令 P[s] 为和为 s 的所有序列数。枚举最后一枚面值 a，前面的序列数为 P[s−a]，把各面值贡献相加。金额在外从小到大，面值在内，恰好对应这个状态。正面值确保依赖更小的和。

对最大值运算，同一方案重复出现有时不会改变数值；对加法计数，重复一次就多算一次。因此“交换两层循环结果还对”不能从最值题推广到计数题。

> **相同与取模**
>
> 若题意按面值区分种类，同面值重复输入一般需按题意合并；若每个编号都是不同物品，不能随意去重。计数可能非常大，题目要求取模才取模；模数与中间加法范围都要检查。

> **停下来，轮到你想**
>
> 用面值 1、2 和目标 3，分别写出组合名单与序列名单。再为两个模型各说一句状态定义，解释每种方案为什么恰好被统计一次。

这一站的学习证据：先定义方案是否有序，再解释循环；不会先背两层循环后猜统计对象。

库存有限：多重背包与二进制拆分

这一站带走：先枚举数量，再把一批物品打成若干包。

先修：第 9、12、14 站

不是一件，也不是无限件，而是最多 s 件。

第 i 种有 s_i 件，每件重量 w_i、价值 v_i。先从最直接的定义出发：这类选 k 件，$0 \le k \le \min(s_i, \lfloor c/w_i \rfloor)$，来源必须是上一种物品处理完的旧行。

> $F[i][c] = \max_{\text{合法 } k}\{F[i-1][c-k\cdot w_i]+k\cdot v_i\}$

朴素时间可写成 $O(W\cdot\Sigma \min(s_i, \lfloor W/w_i \rfloor))$。如果数量很小，这个版本简单、可核对，不必为了“高级”立刻优化。

把 13 件分成 1、2、4、6 四个包

每包只能整体选一次。选大小为 k 的包，相当于新增一个 0/1 物品，重量 k·w、价值 k·v。前面的 1、2、4 能组合出 0 到 7；加上最后 6 又能覆盖 6 到 13，合起来正好覆盖所有允许数量，且总包大小只有 13，不会超库存。

一般地，若已有包能表示 0…R，下一个包大小 $b \le R+1$，则不选它能得到 0…R，选它能得到 b…b+R，两个区间连在一起。按 1、2、4…取包，最后接剩余数量，正是用这个覆盖性质。拆后做 0/1 背包，时间 $O(W\cdot\Sigma \log(s_i+1))$，先把过大的库存截到 $\lfloor W/w_i \rfloor$ 还能省事。

若同一道题混着不同库存?

这叫混合背包。每种物品单独按次数限制处理：单件走 0/1 更新，无限库存走完全更新，有限库存先枚举数量或拆包。它们共享同一个容量状态，外层仍一类一类推进；“恰好/至多”的定义和初值必须始终一致。

> **一个提高组陷阱**
>
> 二进制拆分能保持可达与最优值，却未必保持方案数。例如 13 拆成 1、2、4、6，数量 6 既可选“6 包”，也可选“2+4 包”。原题按每种拿几件计数时，会被重复统计。最优值等价，不等于计数等价。

> **停下来，轮到你想**
>
> 把 10 件分成若干 0/1 包，证明从 0 到 10 都能表示且不能超过 10。再找一个数量，检查表示方式是否一定唯一。

这一站的学习证据：能解释拆分覆盖、库存上限与计数限制；有朴素数量枚举作为核对版本。

同组只能选一个：分组背包

这一站带走：把“物品层”改成“组层”，防止同组串用。

先修：第 8—10 站

早餐可以选一种主食，不代表可以把整组菜单都点一遍。

物品分成若干组，每组至多选一件。状态 F[g][c] 表示只处理前 g 组、总重量不超过 c 的最大价值。第 g 组可能不选，也可能选其中的任意一件；不论选谁，都从**前 g−1 组**的结果出发。

> new[c] = max(old[c], 所有该组物品 x 的 old[c−w_x]+v_x)
>
> 其中 $w_x \le c$；old 表示该组开始之前的快照。

若像普通 0/1 背包那样，先完整更新组内物品 A，再完整更新同组 B，即便每次容量都倒序，也可能把 A 与 B 一起选中。因为处理 B 时的 dp 已经含有 A。

最清楚的实现是保存组开始前的 old，只读它；new 负责比较本组所有候选。本章基础时间 O(W·物品总数)，两行空间 O(W)。正重量条件下也可以精心安排原地循环，但在理解组层之前，不急着省这一行。

若每组必须恰好选一件呢?

删除“不选本组”的分支，new 每轮先设不可达，只从组前 old 加一个本组选项。若某容量下没有合法选项，它就不可达。用全零 new，会偷偷容许缺组选项。

> **卡片翻面是必选组**
>
> 一张卡有正反两面，必须选其中一面，可以视为一组两个候选。“每张都要翻出一面”与“可以跳过一张”是不同模型。

> **停下来，轮到你想**
>
> 容量 2，同组两件各重 1、价值分别为 3 和 4。追踪“先 A 再 B、各自倒序”的程序会怎样串用，再用组前快照设计正确更新。

这一站的学习证据：能区分物品层和组层；能独立调整至多一件与恰好一件的初值和分支。

有两个限制：重量与件数都要记

这一站带走：只为影响未来的约束增加状态维度。

先修：第 10、16 站

背包还有另一个规定：最多带 K 件。

两份方案重量与价值相同，但一份已经用了 K 件，另一份只用了 K−1 件；面对下一件好东西，它们的选择空间不同。只记重量已经不足够，还必须记录件数。

可以定义 E[i][k][c]：从前 i 件中选**恰好 k 件、总重量恰好 c**的最大价值。唯一空状态 E[0][0][0]=0，其余不可达。拿第 i 件时，来源为 E[i−1][k−1][c−w_i]；不拿仍来自 E[i−1][k][c]。

如果题目问“至多 K 件、重量不超过 W”，答案要在 0≤k≤K、0≤c≤W 的可达状态中取最大值。也可另定义两个维度都为“至多”的表，但必须同时调整初值与答案位置，不能把两个版本拼在一起。

压缩与成本

压掉 i 后存 dp[k][c]。一种容易证明的安全实现是 k 从大到小、c 也从大到小；读到的 k−1 层仍是本件处理前的状态。时间 O(nKW)，空间 O(KW)。重量、体积两个上限时，类似地得到 O(nWV)。

并不是“有三个数字就开三维”。价值通常是格子里的优化目标，不一定再做一个下标。每多一维，状态数要乘上它的取值范围；写代码前先算乘积。

> **停下来，轮到你想**
>
> 把“总重量≤5、恰好选 2 件、最大价值”写成状态卡。比较“最多 2 件”与“恰好 2 件”的答案读取范围，再估计 n=100、W=1000、K=20 时的工作量。

这一站的学习证据：能证明新增维度必要，能准确说明每个下标和答案集合。

容量太大：把价值搬到横轴上

这一站带走：用“某个价值需要的最小重量”重新设计状态。

先修：第 10、13、17 站

横轴上不一定非得写重量。

如果 n=100、W=10^9，按容量逐格计算就很困难。但若每件价值最多 1000，总价值 V≤100000，价值这一轴反而很短。

定义 G[i][p]：从前 i 件选，总价值**恰好 p**时需要的最小总重量。初始 G[0][0]=0，其余为正无穷。不选沿用上一层；选择第 i 件时，从价值 p−v_i 的上一层最小重量再加 w_i，取较小值。

> G[i][p] = min(G[i−1][p], G[i−1][p−v_i]+w_i)
> 答案：满足 G[n][p]≤W 的最大 p。

为什么只留最小重量足够？对于相同的已处理范围和相同价值 p，重量更小的方案至少同样容易接上任何后续，较重方案就不必为求最大价值而单独保留。这是第 4 站“未来选择不会更差”的同一个理由。

时间 O(nV)，空间 O(V)。0/1 压缩仍需旧层来源；本章采用正整数价值来解释严格递减的来源。若有零价值或其他特殊条件，要据状态另作检查。重量可能累加到很大，表格类型与无穷值需要覆盖总重量，而不只是单件重量。

> **实际题目中的选择**
>
> AtCoder Educational DP Contest 的 D、E 都是 0/1 最大价值。D 的容量上限较小；E 的容量大、总价值小。状态选择来自数据范围，不来自题目名字。题源与约束见第 28 站。

> **停下来，轮到你想**
>
> 有三件物品，重量分别很大，价值分别为 2、3、4。不要按重量画表，改用总价值作横轴；用自己的话说明最后为什么要“找最大可行价值”，而不是直接输出 G[n][W]。

这一站的学习证据：能从约束主动选择较小的一维，并证明新状态保留的信息足够。

不仅要分数：沿着来路找回名单

这一站带走：理解前驱必须属于当时的正确层。

先修：第 7—10 站

表告诉你能有多好，来路告诉你是怎样做到的。

若保存完整二维 0/1 表，从 (n,W) 倒着走。如果 F[i][c] 与 F[i−1][c] 相同，可以选择“不拿第 i 件”，然后令 i 减 1。否则，最优值必须由拿第 i 件而来，把它加入名单，容量减 w_i，再进入上一层。

如果两条来源一样好，它们都能构成最优解。题目允许任意最优名单时，固定选一个合法分支即可；若要求字典序最小，不能仅凭“相等时不拿”就宣称满足，需要另外证明。

恰好模型也可这样追踪，但每次先核对来源可达以及转移等式。对于翻面卡片，应记录这一层来自正面还是反面，倒推到第 0 层，最后反转得到原顺序。

为什么一维数组上随便存一个前驱会出错?

只有一件 (2,3)、容量 4，倒序更新 dp[4] 时记录“来自容量 2”。当时 dp[2] 是旧层的空方案；稍后 dp[2] 又被这件物品更新。若最后沿可变的 pre[4]→pre[2] 追踪，就可能把同一件拿两次。数值计算正确，追溯历史却被覆盖了。

> **可先选稳妥方案**
>
> 初学保留二维表或按层保存选择记录。空间紧时可以研究不可变前驱节点、检查点重算或分治恢复，但要单独核算空间；“额外存一个 pre[c]”不自动等于正确恢复。

> **停下来，轮到你想**
>
> 用只有一件 (2,3)、容量 4 的例子，画出可变前驱被覆盖的过程。然后从二维表恢复一份合法名单，重新计算重量与价值，和最终格核对。

这一站的学习证据：能验证输出名单本身合法，而不只验证最优值正确。

最优方案有几种：分数与计数同时维护

这一站带走：在更好时替换，在相等时累加，并证明不重复。

先修：第 10、14、15 站

有两个同样好的选法，不能只留其中一个数字。

每件物品有独立编号、至多一次，要求重量恰好 W 的最优价值与最优子集数量。对每个状态同时维护 best 和 ways。空状态 best[0][0]=0、ways[0][0]=1；其他重量不可达、ways=0。

合并一个合法候选时：如果候选分数更高，就同时替换 best，并把 ways 设为该来源的方案数；若分数相同，就把来源的 ways 加进来；分数更低则忽略。不可达来源不能参与比较或贡献计数。

为什么相等时可以相加?

对同一物品层，不拿第 i 件与拿第 i 件的子集互不重叠。两类内部又由上一层正确统计，因此相加不会把同一子集数两遍。两件重量价值完全相同、但编号不同的物品，选第一件与选第二件是不同子集；若题目按种类数量区分，则要重新定义计数对象。

若题目改成总重量≤W，可用“恰好重量”表：先求所有 c≤W 中最大 best，再把达到最大值的这些精确重量状态的 ways 相加。每份子集只有一个真实重量，所以只归入一个 c。不要把“至多容量”的 ways 对所有容量相加，一份轻的方案可能在多个上限下反复出现。

> **与上一章呼应**
>
> 二进制拆分可能使同一种数量有多个包表示；能求对最大值，不代表能直接求对最优方案数。涉及计数时，必须再证明一一对应。

> **停下来，轮到你想**
>
> 两件不同编号的物品都为 (1,2)，要求恰好重量 1。自己列最优名单，并手推 best 与 ways；接着把目标改成重量≤2，解释哪些状态可以相加、哪些不行。

这一站的学习证据：能证明计数分类互斥，知道取模作用于 ways 而非随意作用于 best。

选了孩子必须选父亲：依赖与树上背包

这一站带走：把一棵子树变成一组“花费—收益”选择。

先修：第 10、16 站；树、递归与后序遍历。未学树时留待以后。

有些装备必须依附主机；有些课程必须先学前置课程。

先考虑一棵有根树，每个非根节点只有一个父亲，选一个节点必须选它的父亲。所有真实节点正重量、价值非负。目标仍是重量≤W，最大价值。不能把每个节点当独立物品；单独选一个便宜高分的孩子可能不合法。

定义 `f[u][c]`：在 u 的子树中，**必须选 u、总重量恰好 c**时的最大价值。先只选 u，因此 f[u][w_u]=v_u，其他为不可达。再逐个合并孩子的子树。

两张表怎样合起来?

设 old 是 u 已合并部分孩子后的表，下一个孩子为 x。可以整棵不选 x 的子树，直接继承 old；也可以给 x 子树分 t 容量，读取 f[x][t]，它已保证选了 x，并满足其内部依赖。合并候选是 old[c]+f[x][t]，写入 new[c+t]。

> new = old（表示跳过整个孩子子树）
>
> 对可达 old[c] 与可达 f[x][t]：
>
> new[c+t] 与 old[c]+f[x][t] 比较，要求 c+t≤W。

要用合并前的 old 快照，防止同一孩子子树被使用多次。子树求好以后再算父亲，这叫后序计算。每个合法方案在各孩子子树上的分配唯一；把所有容量划分都枚举到，就不会漏掉跨子树的最优搭配。

对多棵互不相关的树，可加一个重量与价值均为 0 的虚拟根，把各树根接上，强制选虚拟根。真实节点仍为正重量。若要输出“选课数”，虚拟根不能误计成真实课程。

> **提高深度与边界**
>
> 朴素每次合并枚举两个容量，保守时间 O(nW²)、空间 O(nW)，可用子树实际可达范围减少枚举；按节点数量作容量的特例可有更紧界，不能直接套到任意重量。一个节点依赖多个父亲时不再是本章树模型，也不能用此递推硬套。

> **停下来，轮到你想**
>
> 画根 A 和两个孩子 B、C，另给 B 一个孩子 D。列出“不选 B 却选 D”为何非法，再写出合并 C 子树时 old 与 f[C] 分别允许哪些节点。

这一站的学习证据：能独立定义强制选根的状态，正确处理跳过孩子、虚拟根与容量拆分。

多重背包再提速：单调队列优化

这一站带走：从数量枚举中提取滑动窗口最大值。

先修：第 15 站；余数分组、双端队列与单调队列。此站为提高选学。

先看清每次在重复比较什么，再决定需要哪种数据结构。

固定一种物品：重量 w、价值 v、最多 s 件。old 是处理本种之前的容量表。直接枚举取 k 件：new[c]=max(old[c−kw]+kv)。每个容量都重新比较许多 k，工作量大。

第一步：按余数分开

从 c 不断减 w，余数不变。因此把容量分成 r、r+w、r+2w…，其中 0≤r<w。固定一组，写 c=r+qw。令前驱位置为 r+tw，则 k=q−t，库存限制变成 max(0,q−s)≤t≤q。

> $new[r+qw] = qv + \max_{\max(0,q-s)\le t\le q}\{old[r+tw]-tv\}$

qv 与候选 t 无关，可提到外面。每个候选的分数 H[t]=old[r+tw]−tv 固定不变；q 每前进一格，可用 t 的范围就右移一格。问题于是变成长度至多 s+1 的滑动窗口最大值。

第二步：只留下仍可能当冠军的候选

队列存 t，位置递增、H 分数递减。每个 q：先从队首删除 t<q−s 的过期候选；再考虑新候选 t=q，从队尾删除分数不大于它的旧候选，再将它入队；最后用队首最大 H 加 qv。分数相等时保留较新的候选，因为它过期更晚。

较新的候选若分数还更高，旧候选无论在哪个未来窗口里都不能胜过它，并且会更早过期，所以可以永久删掉。每个候选最多进出各一次，每种物品合计处理 O(W) 个容量，总时间 O(nW)、空间 O(W)。

例如 w=2、v=3、s=2，固定偶数容量。old 在 0、2、4、6 的值为 0、4、5、9，对应 H 为 0、1、−1、0。到 q=2，窗口 t=0…2，最大 H 为 1，所以容量 4 的新值是 2×3+1=7。t=1 意味着旧容量 2，加当前物品一件。

> **不能丢的检查**
>
> 候选必须读 old，不能读正被改写的 new。恰好型中不可达 old 不进队；队空表示本状态不可达。窗口包含两端，共 s+1 个候选；把 q−s 删除也会少算“恰好拿 s 件”。

> **停下来，轮到你想**
>
> 先不写队列，用 w=2、v=3、s=2 的小表直接枚举 k；再换成窗口表达式逐格对照，说明 t、q−t 分别代表什么。两种结果一致后，再自己实现队列。

这一站的学习证据：能独立推导变换与候选删除依据；优化版必须与朴素数量枚举对照。

许多真假一起算：bitset 子集和

这一站带走：理解移位与按位或为何对应拿或不拿。

先修：第 11 站；二进制位、移位与按位或

一排小灯，一次就能平移并合并。

对子集和，把可达总和 s 记为第 s 位的一盏灯。初始只有第 0 位亮。处理一个数 a 时，旧的所有亮灯向高位平移 a，就代表“每份旧方案都再拿这张卡片”；与旧灯做按位或，保留不拿与拿两类结果。

> 新灯 = 旧灯 OR（旧灯向高位移 a 位）

卡片依次为 2、3，灯最初在 {0}，处理 2 后在 {0,2}，再处理 3 后在 {0,2,3,5}。平移使用的是整排旧灯，因此一轮只新增一张卡片。C++ 的 `bits |= (bits << a)` 就表达这项局部操作，但容量大小、初始化、输入与输出仍由你依据题意设计。

这里按位或合并的是真假，不能直接用它求最大价值或方案数。计数也不是把灯相加就自动正确。

速度来自机器并行处理很多位

若位集按 b 位机器字打包，常见实现一次移位/或约需 $O(\lceil (S+1)/b \rceil)$ 个字操作，n 轮约 $O(n\lceil (S+1)/b \rceil)$，空间约 S+1 位。具体库实现和机器决定实际速度；不是把一次大位集操作算成 O(1)。

> **容量裁剪的依据**
>
> 只保留 0…S 的位，依赖所有数为非负：超过 S 的和以后不会再变小。若允许负数，就不能直接裁掉。固定大小 bitset 的长度必须覆盖目标索引，移位丢掉高位后不会自动找回来。接口语义可查 C++ 工作草案 bitset。

> **停下来，轮到你想**
>
> 用卡片 [2,2,5] 画出每轮亮灯位置。解释为什么两张不同的 2 可以凑出 4，而处理第一张时还不行；然后判断“灯亮”是否告诉你有几种凑法。

这一站的学习证据：能把位运算还原为状态转移，知道加速的是哪个维度以及不适用哪些目标。

件数少、数值大：折半搜索

这一站带走：换一条复杂度路线，而不是硬开巨大容量表。

先修：第 3、4、18 站；排序、二分与子集枚举

40 件物品难以全枚举，但各枚举 20 件就轻得多。

当重量和价值都很大、n 约 40 时，可以把物品分成左右两半。分别列出每半所有子集的 (重量,价值)，去掉超过 W 的。每边约 2^{20} 个子集，而不是枚举 2^{40} 份完整名单。

右半怎样帮助左半找搭档?

按右半总重量从小到大排序，维护前缀最大价值。对左半某方案 (wL,vL)，剩余容量 R=W−wL；二分找到右半重量≤R 的最后位置，读取这个前缀的最大价值，就得到了它的最佳搭档。遍历左半所有方案，取最佳合并结果。

这里删除右半“更重且不更值钱”的方案是安全的：它与更轻、更值钱的方案使用的是同一半物品，与左半的兼容关系相同。但不能据此在普通 0/1 输入里随意删除一个被另一件支配的单件，因为两件也许可以同时选择。

排序和逐个二分的一种常见实现时间 O(n·2^(n/2))，存储 O(2^(n/2))；n 为奇数时按两半大小取上界。约 2^{20} 对数据本身就需不少内存，还要计算容器、排序与运行时开销。

> **为什么它不漏**
>
> 每份完整子集都能唯一拆成左半子集与右半子集。枚举左半时只需找到其合法右半搭档中最好的一份；其他搭档不可能让这份左半更好。

> **停下来，轮到你想**
>
> 给 6 件物品分成两半，各列子集。任选一份左半名单，标出右半所有合法搭档，再解释“按重量前缀最大”为什么能直接回答它。

这一站的学习证据：能从 n 很小、数值很大识别折半机会，并证明两半合并不漏解。

提高组的第一步：看范围，选状态

这一站带走：把建模、复杂度和实现边界放在同一张地图上。

先修：第 7、15、18、21—24 站按需选学

最熟的模板，不一定是这道题最合适的模型。

观察到的条件	候选路线	常见时间
n 很小	全子集枚举	$O(2^n)$，按实现可多一个 n
W 或总重量较小	按重量/容量的 DP	O(n·min(W,Σw))
总价值 V 较小	按价值的最小重量 DP	O(nV)
n 约 40，数值很大	折半搜索	O(n·2^(n/2))
只问正数子集和是否可达	bitset 加速	约 O(nS/b) 字操作
每种有较大库存	二进制拆分或单调队列	O(WΣlog(s+1)) 或 O(nW)
有分组或树依赖	组层/子树合并	先按实际转移次数估计

O(nW) 里的 W 是容量的数值，不是输入文件的长度。用几十个二进制位就能表示很大的 W，表却要开 W+1 格。这类按数值大小计时的算法称为**伪多项式算法**。知道这个词的用途，是避免说“有多项式就一定能跑”。

估算时写清状态数×每格候选数。n=200、W=200000 是约四千万次基本状态访问；若每格又枚举 100 个数量，就会再乘 100。机器、语言、实现常数和时限都会影响能否通过，不能用一个固定的“每秒次数”保证结果。

把背包当作状态设计训练

模型的三根主线是：按什么层处理选择；用什么下标记住未来需要的约束；格子里优化或统计什么。变化后的问题也许不叫背包，却仍能通过“按最后一次决定分类”建立 DP。

提高组深度并不意味着给每种名字记一个模板。树上合并、价值维度、单调队列分别解决的是依赖结构、状态数量、转移成本。先指出瓶颈，再选工具。

> **停下来，轮到你想**
>
> 为三组范围各选一种起步方案并说明依据：① n=25、重量很大；② n=100、W=10^9、$v_i \le 100$；③ n=1000、W=2000。先估算工作量，再决定是否需要进一步优化。

这一站的学习证据：能说清状态充分性、时间与空间，不把“看起来是背包”当作最终算法判断。

调试：让错误在最小例子里现形

这一站带走：用专门区分模型的测试找第一次偏差。

先修：第 7—20 站按练习选择

样例通过只是起点，反例才会告诉你哪里理解错了。

怀疑的问题	最小区分输入	追踪重点
0/1 写成无限次	单件 (2,3)，W=4	来源格是不是本轮刚更新的
恰好误用全零初值	单件 (2,3)，目标 3	重量 1 的来源是否真实可达
同组拿了两件	同组 (1,3)、(1,4)，W=2	来源是否仍是组前快照
计数混淆组合与序列	无限面值 1、2，目标 3	1+2 与 2+1 是否应区分
必须选够却允许空方案	恰好选一件，唯一收益 −5	0 是否来自非法的“不选”
可变前驱破坏名单	单件 (2,3)，W=4	同一编号是否重复出现
整数范围不足	多件价值 10^9 且都能装下	中间值与最终和用什么类型

先分类：编译错误看类型与语法；卡住看控制量是否推进；崩溃看边界与内存；数字不对看状态、初值、来源和输出；超时再看工作量。不要同时重写所有部分。

小规模暴力是你的裁判

自己保留一个 n≤15 的全子集枚举程序。产生小重量、小价值的随机数据，同时运行暴力与 DP，比较最优值。多重背包优化应与枚举数量版本比较；组与树模型的暴力也必须检查组限制和依赖，不能用一个漏掉约束的“裁判”。

出现不一致后，删去物品、缩小数字，直到留下最小反例。打印每轮的关键状态，定位第一次与手推不同的格子。先改根因，再重新跑关键区分、极端边界、最小情况三类测试。

> **额外性质检查**
>
> 在本讲非负价值、允许不选的至多模型中：容量增大答案不能下降；增加可选物品答案不能下降；答案不超过全部价值和。恰好模型、负收益或必选组不能直接照搬这些性质。

> **停下来，轮到你想**
>
> 从表中任选一种错误，先故意写出它，再用最小反例抓住第一次偏差。修正后，用一句完整的话解释防错动作，而不是只记“循环写反了”。

这一站的学习证据：能给出失败现象、第一次偏差、根因、修法和区分新旧模型的证据。

独立远征：三道逐层融合的任务

这一站带走：让孩子独立建模，实现与验证，不照抄现成题解。

先修：A：第 1—10、19 站；B：第 10、16 站；C：第 21 站

下面不给模板。选择与你当前能力相邻的一题，完成一个闭环。

远征 A：营地装备单

有 n 件独立装备，每件最多一次，重量 w、价值 v。总重量≤W，求最大总价值；进阶要求再输出一份编号名单。输入第一行 n W，之后 n 行 w v。1≤n≤100，1≤W≤2000，1≤w≤3000，0≤v≤10^6。可以不选任何装备。

自己设计四件的小数据并穷举核对；再测试全部超重、恰好装满与不同名单同分。先完成数值版，确认稳定后才增加名单恢复。

远征 B：每站必须带一种补给

有 g 个补给站，每站给出若干方案，每个方案重量 w、快乐值 v。必须从每站恰好选一个方案，总重量必须恰好 W，求最大快乐值；无解输出 Impossible。输入 g W；每站先给方案数 m，接着 m 行 w v。1≤g≤30，1≤m≤8，1≤W≤1000，1≤w≤1000，−100≤v≤100。

先用两站、每站两个方案检查模型，再试某站所有选择都会超重、目标不可达、最佳值为负数。注意“负数答案”与“无解”不是同一件事。

远征 C：带前置条件的研究项目

n 个项目构成一片森林，选项目 i 必须选它的父项目 p_i（0 表示无父亲）。每个项目耗时 w_i、收益 v_i，各做最多一次，总耗时≤W，求最大收益。输入 n W；之后 n 行 p_i w_i v_i，保证无环且 $0\le p_i<i$。1≤n≤80，1≤W≤200，$1\le w_i\le 200$，$0\le v_i\le 1000$。允许不做项目。

只在已学树与第 21 站后挑战。先画三四个节点的树，穷举合法集合；核对虚拟根、跳过整棵子树、多个根共享预算的情况。

> **三道题都提交同一套证据**
>
> 状态完整句、初值理由、转移分类与来源层、复杂度估算、关键/边界/最小三类测试、孩子自己的实现。先尝试，卡住时再由爸爸调用下一条提示；不要求一次完成三题。

> **停下来，轮到你想**
>
> 任选一道远征，先完成模型卡，不看前文模板。向爸爸解释“我保留的这些信息为什么足够”，爸爸听完后再决定是否需要提示。

这一站的学习证据：能在改变题面、负收益和组合条件后独立判断，遇到失败能主动用小反例定位。

回到 AtCoder：按方法选题，不按字母猜难度

这一站带走：用核实过的原题搭桥，给每题安排一个学习目标。

先修：按表中对应站点；原题需要联网阅读

背包学完一小段，就用一题检验；不等整本看完才开始写。

以下是方法训练路线，不是官方难度评级，也不是必须刷完的题单。链接与所列约束已于 2026-10-12 核对。每次选择当前层级的一题，先独立读原题；这里不提供可提交代码。

原题	适合接在哪一站	本次只抓一个目标
Educational DP D · Knapsack 1 **$N\le100$，$W\le10^5$，$v_i\le10^9$**	第 9 站后	0/1 状态、方向、64 位价值和；约 O(NW)
ABC286 D · Money in Hand **$N\le50$，$X\le10^4$，每种数量≤50**	第 11、15 站后	有限库存能否恰好付款；先枚举数量再考虑优化
ABC271 D · Flip and Adjust **$N\le100$，$S\le10^4$**	第 16、19 站后	每张必须选正反一面，恰好总和并恢复方案
ABC015 D · 高橋くんの苦悩 **$N\le50$，$W\le10^4$，$K\le N$**	第 17 站后	宽度与张数两个限制；约 O(NKW)
Educational DP E · Knapsack 2 **$N\le100$，$W\le10^9$，$v_i\le1000$**	第 18 站后	主动换成价值轴；O(NΣv)
ABC032 D · ナップサック問題 **N≤200；另有三类约束保证之一**	第 24—25 站后	根据 n、重量和、价值和选择算法；注意输入是 v w
ABC267 D · Index × A (Not Continuous ver.) **$M\le N\le2000$，A_i可负**	迁移选学	恰好选 M 项且保留原顺序；收益依赖已选数量，重新设计状态

ABC032 D 的三类条件至少一个成立：N≤30，或所有重量≤1000，或所有价值≤1000。先读完约束再选方法；不要因为题名是背包就一律按 W 开表。ABC267 D 的收益与选出后的位置有关，是从背包思路迁移到序列 DP，不是直接复制普通固定价值模板。

本次 ABC 479 B 应怎样接回来?

ABC479 B · Longest Rainy Streak 要求最长连续雨天，N≤100，本身不是背包。可以从“变量应记录什么、怎样根据读到的新字符更新”连接到本讲状态思维，但它没有逐件拿或不拿、分配容量的结构。

家长提供的事实：孩子在爸爸讲解后自己完成并 AC；C 题基本没有时间阅读。尚未查看孩子原始错误代码，不能把 B 的卡点诊断为缺背包，也不能从没时间看 C 推断不会 C。隔一两天先让孩子独立重做 B，保留思路与测试证据即可，不把答案重讲一遍。

> **停下来，轮到你想**
>
> 从题单中选择一题，先只写“限制、选择次数、目标、状态句、估计状态数”。若这五项写不清，回到对应站手推，不急着开代码。

这一站的学习证据：能将方法迁移到真实题面；记录哪些步骤独立完成，哪些用了提示。

给爸爸：把你的理解变成她的思考空间

这一站带走：用分层验收和完整任务陪练，保护独立解决问题的过程。

先修：家长阅读；无需让孩子一次看完全部教学安排

爸爸的技术经验，用来判断她卡在哪一层；关键选择与代码留给她完成。

这份讲义覆盖到提高组常见的状态设计与优化，不是一轮课程必须全部学完的清单。孩子已通过 GESP C++ 四级，算法学习仍需依据实际表现分层。“从 0”指从背包概念与建模出发，语法前提是基本数组、循环、条件与整数运算；更难工具在对应站标了先修。

阶段	建议范围	继续前要看到的证据
第一轮：建立模型	01—10；一站可拆多次	能独立定义状态、推导 0/1、解释方向与恰好初值
第二轮：改变条件	11—20，按需求选学	不看模板，能随次数、目标、分组改变状态
第三轮：提高选学	21—25；每次只选一条支线	先修树/队列/位运算等已稳；能证明优化等价
回到实战	26—28 按当前层级选	能独立实现、设计反例、隔日解释并迁移

第一节可以直接这样开始

准备三张装备卡与容量 6 的纸背包。说：“每件最多一件，我们要让总价值尽量高。请你设计一种不漏方案的比较办法，再用它选出一份名单。”让孩子完成一个完整任务；若她已经会枚举，就直接进入“怎样把同样的子问题合在一起”，不要继续拆成加法填空。

首次可用 20—35 分钟作为观察起点，纳入原有学习时间；不额外堆题。能解释一条关键关系、愿意下次继续，就可以收尾。出现疲劳时只保存手推与待解问题，不用高级知识冲淡已有成就。

听到不同回答时，怎样推进?

孩子的表现	先确认什么	下一项完整任务
只选单件价值最高	会比较单件价值	用第 2 站三件物品比较完整组合
会枚举但不会 DP	模型和合法性已建立	给两个后续完全相同的分支贴状态标签
能写转移，方向靠背	二维分支正确	用单件反例画新旧层并独立决定更新顺序
会做至多，不会恰好	容量轴已理解	比较无物品时每个状态是否真实存在
听懂、AC，但隔天写不出	已有一次辅助完成经历	换一个小题面，独立走模型到自测闭环

诊断话术与提示边界

先完整看题与代码，在心里找主根因和最小反例。开口先说已正确部分，再指出核心偏差所在，例如：“你把拿与不拿分开了，这一步成立；现在一维更新读到了本轮的结果，同一件可能重复出现。请用只有一件物品的例子追踪，再自己设计修改。”说完等待，不替她接着写。

若确实卡住，再缩到一个来源格或提供局部骨架。提示后让她独立完成下一步，并换数据检验。技术名词、漂亮代码与更快算法都不抢在理解前面。

比赛与专题怎样并行?

ABC 继续作为观察读题、实现与节奏的窗口。赛后分开记录：建模不会、实现偏差、测试不足、时间分配。共同参赛时提前约定谁读题、谁动键盘、什么时候求助；记录提示参与程度，才看得清孩子自己的成长。背包只是其中一个专题，不承诺学完即可解决每场 B、C。

> ### 停下来，轮到你想
>
> 第一次陪练只选一个目标。结束时请孩子用自己的话解释依据，记录“独立完成/提示后完成/仍待解决”的事实，不代写她的反思。

这一站的学习证据：保持兴趣、有效思考与能力迁移三条底线；不把阅读进度或答题勾选当作掌握率。

带走一张方法卡：下一题从哪里开始?

这一站带走：把知识点收束成可以反复使用的解题与复盘动作。

先修：完成当前学习层级即可；无需先学完提高选学

当你又遇到一堆选择，先把问题说清，再决定是否需要背包。

步骤	我要自己回答
读题	选的是什么？每件几次？能否拆开？有组或依赖吗?
目标	至多还是恰好？求最大、最小、能否、计数还是名单?
手推	用两三件列方案；一个反例能否区分两个模型?
状态	允许前几件/几组？下标的单位是什么？格子里是什么?
转移	按什么决定分情况？是否不漏不重？来源在哪一层?
初值	空方案在哪里合法？不可达如何表示?
实现	怎样保证依赖已算好？压缩会不会改变来源版本?
成本	状态数×候选数有多大？数值类型与空间够吗?
答案	读哪格或哪些格？无解如何表示？名单能否验证?
自测	关键区分、极端边界、最小输入；小规模暴力核对

思路卡：卡住当天也可以写

这题在讲什么：_____

我已经能解释的部分：_____

我手推的一个小例子：_____

现在卡住的一个问题：_____

下次先检查：_____

改对或 AC 后，再写正式反思

我错在哪里：_____

为什么会这样想或这样写：_____

下次用什么具体动作检查：_____

能区分旧思路与新思路的测试：_____

以上由孩子自己填写。爸爸或教练的判断另记，不替换孩子原话；尚未改对时，只记“待复盘”。AC 之后再用新小例子追踪一次，解释输出为什么正好回答题目。

> **这一程真正要带走的能力**
>
> 把过去压缩成足够的信息；按最后一步分类；让每个状态都有真实含义；证明存储优化没有改变模型；用反例与测试检验自己的判断。遇到新问题，这些能力比任何一份模板更耐用。

> **停下来，轮到你想**
>
> 合上讲义，为一道新的选择题写自己的方法卡。只要能清楚解释下一步该查什么，即使暂时没做完，也留下了可以继续思考的入口。

这一站的学习证据：能用自己的语言迁移方法，既知道已解决什么，也知道还不确定什么。

我的手推、解释与待复盘